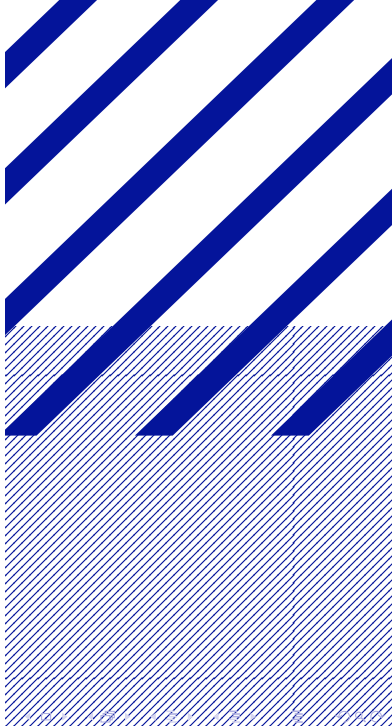




Laufzeitvergleich Gauß-Seidel- Verfahren

Jan Kruse

25. Mai 2026



Spektralradius $\rho(M)$

Größter Eigenwert < 1

Problem: Eigenwerte berechnen zu rechenintensiv

Diagonaldominanz

Zeilensummenkriterium berechnen:

$$\sum_{i=1, i \neq j}^n \frac{|a_{ij}|}{|a_{ji}|} < 1$$

Summen können schnell berechnet werden und die positive Definitheit bestätigt werden

Symmetrie

Symmetrie wird bestätigt durch:

$$A = A^T$$

- Gauß-Seidel-Verfahren sogenanntes Splitting-Verfahren
- Ausgangsmatrix A wird in kleinere Probleme zerlegt

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}$$



$$L = \begin{pmatrix} 0 & 0 & 0 & 0 \\ a_{21} & 0 & 0 & 0 \\ a_{31} & a_{32} & 0 & 0 \\ a_{41} & a_{42} & a_{43} & 0 \end{pmatrix} \quad D = \begin{pmatrix} a_{11} & 0 & 0 & 0 \\ 0 & a_{22} & 0 & 0 \\ 0 & 0 & a_{33} & 0 \\ 0 & 0 & 0 & a_{44} \end{pmatrix} \quad R = \begin{pmatrix} 0 & a_{12} & a_{13} & a_{14} \\ 0 & 0 & a_{23} & a_{24} \\ 0 & 0 & 0 & a_{34} \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Rechenvorschrift im Code:

```
# Inverse berechnen
inverse = np.linalg.inv(D + L)

while r >= accuracy:
    # Gauss-Seidel-Verfahren
    x_m_new = -inverse @ R @ x_m + inverse @ b
    # Berechnung des Residuums
    r = np.linalg.norm(b - A @ x_m_new)
    # Alten Iterationschritt ueberschreiben
    x_m = x_m_new
```

$$x_{m+1} = -(L + D)^{-1} R x_m + (L + D)^{-1} b$$

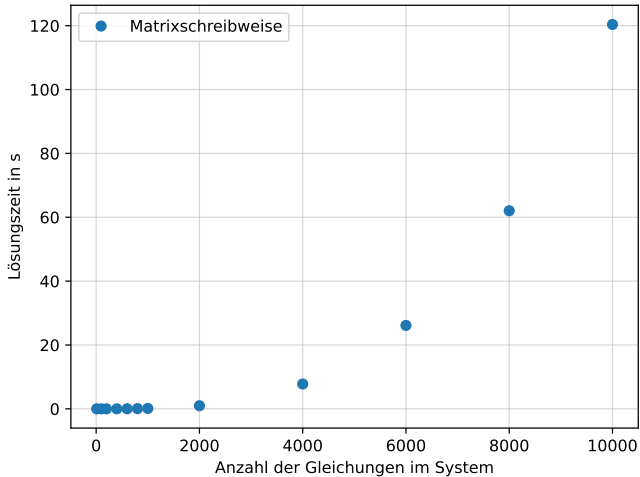


Abbildung: Laufzeiten bei Matrixschreibweise

Summenlogik

$i = 1 \quad x_{1,1} \text{ berechnen}$

$$\begin{pmatrix} 10 & 2 & -1 & 1 \\ 1 & 8 & 2 & -2 \\ -1 & 3 & 12 & 4 \\ 2 & -1 & 3 & 15 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 18 \\ 19 \end{pmatrix} \quad x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad x_{m+1} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$x_{1,1} = \frac{1}{a_{11}}(b_1 - a_{12}x_{02} - a_{13}x_{03} - a_{14}x_{04})$$

$i = 2 \quad x_{2,1} \text{ berechnen}$

$$\begin{pmatrix} 10 & 2 & -1 & 1 \\ 1 & 8 & 2 & -2 \\ -1 & 3 & 12 & 4 \\ 2 & -1 & 3 & 15 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 18 \\ 19 \end{pmatrix} \quad x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad x_{m+1} = \begin{pmatrix} 1,2 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

$$x_{1,2} = \frac{1}{a_{22}}(b_2 - a_{21}x_{11} - a_{23}x_{03} - a_{24}x_{04})$$

$i = 3 \quad x_{3,1} \text{ berechnen}$

$$\begin{pmatrix} 10 & 2 & -1 & 1 \\ 1 & 8 & 2 & -2 \\ -1 & 3 & 12 & 4 \\ 2 & -1 & 3 & 15 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 18 \\ 19 \end{pmatrix} \quad x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad x_{m+1} = \begin{pmatrix} 1,2 \\ 0,98 \\ 0 \\ 0 \end{pmatrix}$$

$$x_{1,3} = \frac{1}{a_{33}}(b_3 - a_{31}x_{11} - a_{32}x_{12} - a_{34}x_{04})$$

$i = 4 \quad x_{4,1} \text{ berechnen}$

$$\begin{pmatrix} 10 & 2 & -1 & 1 \\ 1 & 8 & 2 & -2 \\ -1 & 3 & 12 & 4 \\ 2 & -1 & 3 & 15 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 18 \\ 19 \end{pmatrix} \quad x_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad x_{m+1} = \begin{pmatrix} 1,2 \\ 0,98 \\ 1,36 \\ 0 \end{pmatrix}$$

$$x_{1,4} = \frac{1}{a_{44}}(b_4 - a_{41}x_{11} - a_{42}x_{12} - a_{43}x_{13})$$

Summenlogik

- Ein Summenausdruck kann als Skalarprodukt interpretiert werden
- Da die Vektoren immer zusammenhängen kann mit Slicing gearbeitet werden

```
import numpy as np

x[i] = (b[i] -
np.dot(A[i, :i], x[:i]) -
np.dot(A[i, i+1:], x[i+1:])) / A[i, i]
```

In der Summe der aktuellen Iteration muss nicht $i-1$ geschrieben werden → bei `:i` ist das i exklusiv

Anstelle von `np.dot` hätte auch `@` verwendet werden können

- Abbruchkriterium durch Genauigkeit bestimmt
- Genauigkeit wird als Funktionsparameter übergeben

```
def gauss_solver_index(A, b, x_0, accuracy:int = 1e-4)
```

- while-Schleife läuft bis das Residuum r die Genauigkeit unterschreitet
- Probe durch Berechnung in jedem Schritt von r
- Residuum wird durch den Betrag von r ausgedrückt

```
r = np.linalg.norm(b - A @ x)
```


- Bestimmung der Laufzeit über `time.time()`
- Differenz aus Start- und Endzeit in Sekunden
- Temporäres speichern in Dictionary, später als CSV
- Plot in separatem Skript, um nicht die Berechnung jedes mal ausführen zu müssen

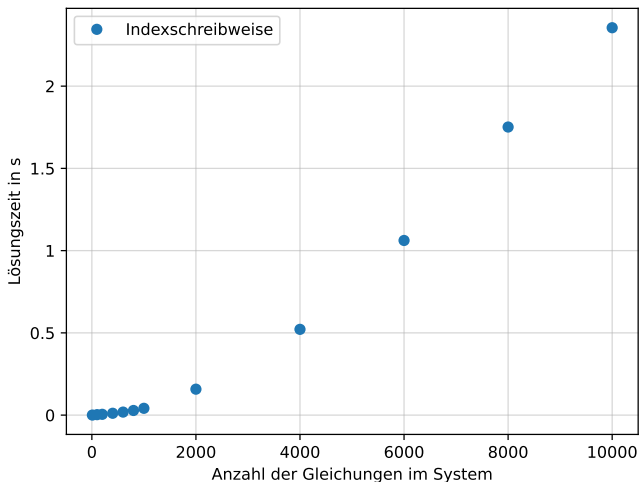


Abbildung: Laufzeiten Indexschreibweise

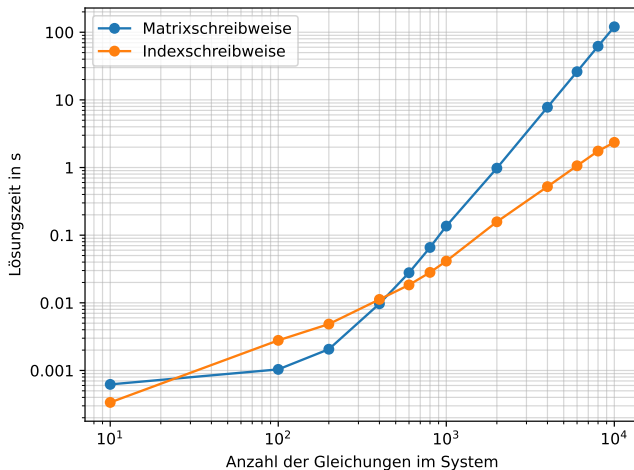


Abbildung: Laufzeitenvergleich

- Indexschreibweise bei größeren Matrizen deutlich schneller
- Vorteile:
 - Es muss keine Inverse berechnet werden
 - Es wird in der for-Schleife jedes Element von x sofort ersetzt
 - Es müssen keine ganzen Matrizen im RAM gespeichert werden
- Matrixschreibweise bei kleineren Matrizen bis $n = 600$ maginal schneller